



MAKERERE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATICS TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

Superstore Sales Analytics System

A Data Warehousing and Business Intelligence Solution

A Project Report submitted to the Department of Computer Science
in Partial Fulfillment of the Requirements for the Award of the

Degree of Bachelor of Information Technology
of **Makerere University**

Supervisor

Dr. Yasin Magombe
Department of Computer Science
ymagombe23@gmail.com

GROUP MEMBERSHIP

SN	Name	Reg. No.	Student No.	Email
1	Emmanuel Kenyi Samuel	23/X/26332/PS	2300726332	emmanuelkenyisamuel@gmail.com
2	Shima Joshua	23/X/26340/PS	2300726340	shimajoshua57@gmail.com
3	Nakanjako Teddy Shillah	23/U/25931/PSA	2300725931	teddyshillah1@gmail.com

May 2026

Declaration

We, the undersigned, declare that this project titled:

“Superstore Sales Analytics System: A Data Warehousing and Business Intelligence Solution”

is our original work and has not been submitted to any other institution. Where the work of others has been used, it has been properly acknowledged in accordance with IEEE citation standards.

Name	Signature	Date
1. Emmanuel Kenyi Samuel		
2. Shima Joshua		
3. Nakanjako Teddy Shillah		

Supervisor's Approval

This project report titled:

“Superstore Sales Analytics System: A Data Warehousing and Business Intelligence Solution”

has been submitted with my approval as the university supervisor.

Supervisor Name: _____

Department: _____

Signature: _____

Date: _____

Dedication

*This work is dedicated first and foremost to the Almighty God,
whose grace, wisdom, and strength sustained us throughout this journey.*

*To our parents and families,
whose unwavering love, sacrifice, and encouragement
have been the foundation of everything we have achieved.*

*To our supervisor and lecturers,
who invested their time and knowledge to guide us
and challenge us to think beyond the ordinary.*

*And to every organisation and professional
working to harness the power of data and technology
to build a smarter, more efficient, and prosperous world.*

— Emmanuel Kenyi Samuel, Shima Joshua & Nakanjako Teddy Shillah

Acknowledgement

First and foremost, we give glory to God for the wisdom, strength, and perseverance that enabled us to complete this project successfully.

We would like to express our deepest and most sincere gratitude to our supervisor, **Dr. Yasin Magombe**, for his exceptional guidance, patience, and invaluable feedback throughout the entire duration of this project. His expertise in Data Warehousing and Business Intelligence greatly shaped our understanding of the subject and challenged us to produce work of the highest standard. We are truly grateful for his time and dedication.

We also wish to extend our sincere appreciation to the **Department of Computer Science at Makerere University** for providing an enabling academic environment, quality instruction, and the resources necessary to undertake this project. The knowledge gained throughout our programme has been instrumental in the successful completion of this work.

Special thanks to the open data community on **Kaggle** for making the Superstore Sales Dataset publicly available. This dataset formed the practical foundation of our analysis and allowed us to apply real-world Business Intelligence techniques in a meaningful context.

We also acknowledge our fellow students and colleagues for the collaborative discussions, constructive feedback, and moral support shared throughout this academic journey.

Finally, we owe an immeasurable debt of gratitude to our **families** for their unconditional love, encouragement, and sacrifice. Their unwavering support during the most demanding periods of this project kept us motivated and focused.

*Emmanuel Kenyi Samuel, Shima Joshua & Nakanjako Teddy Shillah
Makerere University, May 2026*

Abstract

This project presents the design and implementation of a Business Intelligence (BI) solution for a retail superstore. The primary objective is to transform raw transactional sales data into meaningful business insights. The project utilises the Superstore Sales Dataset (9,994 transactions) from Kaggle. A Star Schema data warehouse was designed with four dimension tables and one fact table. The ETL process was implemented using Python 3.14 and Microsoft Power BI was used for interactive dashboards presenting KPIs including Total Revenue (\$2.3M), Total Profit (\$286K), and Profit Margin (12.5%). Key findings show Technology products yield the highest margins, the West region leads in profitability, and the Consumer segment accounts for 45% of sales.

Keywords— Data Warehousing, Business Intelligence, ETL, Star Schema, Power BI, Python

Contents

Declaration	ii
Supervisor’s Approval	iii
Dedication	iv
Acknowledgement	v
Abstract	vi
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Scope and Limitations	2
1.5 Report Organisation	3
2 Literature Review	4
2.1 Data Warehousing	4
2.2 Schema Design	4
2.3 ETL Processes	4
2.4 Business Intelligence Tools	5
2.5 Retail Analytics	5
3 Methodology	6
3.1 Research Approach	6
3.2 Data Collection	6
3.3 Data Quality Analysis	6
3.4 Tools and Technologies Used	7
4 System Design and Implementation	8

4.1	Data Warehouse Architecture	8
4.2	Data Flow Diagram (DFD)	9
4.3	Star Schema Design	10
4.4	Entity Relationship Diagram (ERD)	11
4.4.1	Fact Table: fact_sales	12
4.4.2	Dimension Tables	12
4.5	ETL Implementation	12
4.5.1	Phase 1 — Extraction	12
4.5.2	Phase 2 — Transformation	13
4.5.3	Phase 3 — Star Schema Table Construction	14
4.5.4	Phase 4 — Loading into SQLite	15
4.6	Dashboard Development	15
5	Findings, Analysis, and Recommendations	17
5.1	Key Findings	17
5.1.1	Overall Sales Performance	17
5.1.2	Product Category Performance	17
5.1.3	Regional Analysis	17
5.1.4	Customer Segment Insights	17
5.1.5	Top Sub-Categories by Profit	17
5.1.6	Discount Impact	18
5.2	Recommendations	18
5.3	Conclusion	18
	Appendices	20
	Appendix A: Complete ETL Python Script	20
	Appendix B: ETL Console Output	23
	Appendix C: Star Schema Tables Summary	24

List of Figures

4.1	Three-Tier Data Warehouse Architecture — Superstore BI System	9
4.2	Level-1 Data Flow Diagram (DFD) — Superstore BI System	10
4.3	Star Schema — Power BI Model View	10
4.4	Entity Relationship Diagram (ERD) — Superstore Star Schema	11
4.5	Power BI Dashboard — Superstore Sales Analytics (2017)	16

List of Tables

- 3.1 Data Quality Assessment Results 6
- 3.2 Technologies and Tools Used 7
- 4.1 fact_sales Table Structure 12
- 4.2 Dimension Tables Summary 12
- 5.1 Sales and Profit by Product Category 17
- 5.2 Star Schema Tables Summary 24

Chapter 1

Introduction

1.1 Background

In today's competitive retail environment, businesses generate large volumes of transactional and operational data daily. However, raw data alone is not useful unless it is processed and transformed into meaningful information for decision-making [1]. To address this challenge, organizations use Business Intelligence (BI) systems and data warehousing technologies to analyze data, monitor performance, and support strategic planning.

Business Intelligence involves the use of tools, technologies, and analytical methods to convert raw business data into actionable insights. In the retail sector, superstores manage complex operations involving multiple product categories, customer segments, regions, and shipping processes. Analyzing such large datasets manually becomes difficult, making BI systems essential for efficient reporting and decision-making [2].

Data warehousing provides a centralized repository where data from different operational systems is stored and organized for analysis. One of the most common approaches used in data warehouse design is dimensional modeling, particularly the star schema model. A star schema consists of a central fact table containing measurable business data such as sales and profit, connected to dimension tables that provide descriptive information such as products, customers, regions, and dates [2]. This structure improves query performance and supports multidimensional analysis.

Business Intelligence systems provide retailers with valuable insights into sales trends, customer behavior, regional performance, and profitability. Through analytical tools and dashboards, managers can identify high-performing products, monitor operational efficiency, and make informed business decisions. Modern visualization tools such as Microsoft Power BI further enhance data analysis by providing interactive dashboards, reports, and visual analytics.

This project focuses on designing and implementing a Business Intelligence solution for a superstore dataset using dimensional modeling and Power BI. The project demonstrates how raw transactional data can be transformed into a structured data warehouse that supports reporting, analysis, and data-driven decision-making in retail organizations.

1.2 Problem Statement

The retail superstore generates large volumes of transactional data from daily sales, customer purchases, product inventories, and regional operations. Although this data contains valuable business insights, it is currently stored in fragmented operational systems that do not adequately support analytical reporting or strategic decision-making. As a result, management faces difficulties in quickly identifying top-performing and underperforming product categories, analyzing regional sales trends, monitoring customer purchasing behavior, and tracking key financial indicators such as profit margins. The absence of an integrated analytical system limits the organization's ability to transform raw data into actionable information.

In addition, the lack of a centralized data repository and standardized reporting structure results in slow reporting processes, inconsistent data analysis, and reduced visibility into overall business performance. Decision-makers are therefore unable to access timely and accurate insights needed to respond effectively to changing market conditions and customer demands. If these challenges remain unresolved, the superstore risks poor strategic planning, inefficient resource allocation, reduced profitability, and decreased competitiveness within the retail industry.

Therefore, there is a need for a Business Intelligence solution that integrates transactional data into a centralized data warehouse and provides analytical capabilities through reporting and visualization tools. Such a system would support data-driven decision-making by enabling efficient analysis of sales performance, customer trends, regional operations, and profitability.

1.3 Project Objectives

1. To design and implement a Star Schema data warehouse for retail sales data.
2. To develop an ETL pipeline using Python for data extraction, cleaning, and loading.
3. To create interactive Power BI dashboards visualising key business metrics.
4. To identify sales trends, top products, and regional performance patterns.
5. To generate actionable business recommendations based on data analysis.

1.4 Scope and Limitations

This project focuses on the design and implementation of a Business Intelligence solution for a US superstore dataset containing 9,994 transactional records. The analysis covers key business areas including sales, profit, customer segments, shipping modes, regional performance, and product categories such as Furniture, Office Supplies, and Technology. The

project involves data cleaning, transformation, dimensional modeling using a star schema, and the development of analytical dashboards and reports using Microsoft Power BI.

However, the project has several limitations. First, the dataset used is historical and does not include real-time data feeds. Second, external factors that may influence retail operations, such as economic conditions and market competition, are not included. Finally, predictive analytics are limited to trend-based observations and do not involve advanced machine learning or forecasting models.

1.5 Report Organisation

This report is organized into five chapters that collectively describe the development and implementation of the Business Intelligence solution.

Chapter 1 introduces the study by presenting the background, problem statement, objectives, scope, and limitations. Chapter 2 reviews related literature including data warehousing, ETL, star schema design, and BI tools. Chapter 3 describes the methodology including data collection, quality analysis, and tools used. Chapter 4 presents the system design and implementation including the Architecture (Figure 4.1), DFD (Figure 4.2), Star Schema (Figure 4.3), ERD (Figure 4.4), and Power BI Dashboard (Figure 4.5). Chapter 5 presents findings, analysis, and recommendations.

Chapter 2

Literature Review

2.1 Data Warehousing

A data warehouse is a centralized repository designed to support analytical processing, reporting, and decision-making within an organization. Unlike operational databases that are optimized for daily transaction processing, data warehouses are specifically structured for storing large volumes of historical data and enabling efficient analysis. According to Inmon [1], a data warehouse is *subject-oriented, integrated, non-volatile, and time-variant*.

Data warehousing enables organizations to transform raw transactional data into meaningful information that supports strategic and operational decision-making. A key concept in data warehousing is dimensional modeling, which organizes data into fact tables and dimension tables to simplify analytical queries and improve performance [2]. Fact tables contain measurable business metrics such as sales, quantity, and profit, while dimension tables provide descriptive information such as product categories, customer segments, regions, and dates.

2.2 Schema Design

Schema design determines how data is organized and accessed for analysis. The Star Schema [2] consists of a central fact table connected directly to multiple dimension tables, simplifying queries and improving analytical performance. The Snowflake Schema normalises dimensions further but introduces complex joins that reduce query performance [3]. This project adopts the Star Schema for its simplicity, ease of understanding, and faster query execution.

2.3 ETL Processes

ETL (Extract, Transform, Load) prepares data for storage in a data warehouse. Extraction collects data from sources, transformation cleans and converts it into a consistent format, and loading stores the processed data for analysis. According to Kimball and Ross [2], ETL activities consume approximately 60–70% of total development time. Python and the Pandas library [5] were used for data cleaning, transformation, and preprocessing in this project.

2.4 Business Intelligence Tools

Microsoft Power BI provides powerful features for data integration, reporting, and visual analytics [4]. It supports interactive dashboards, drill-down analysis, filters, slicers, and real-time visualization. Power BI also includes the DAX (Data Analysis Expressions) language for custom calculations and KPIs, making it suitable for Business Intelligence and retail analytics projects.

2.5 Retail Analytics

Retail analytics uses data analysis techniques to understand customer behavior, monitor business performance, and optimize retail operations. According to [3], retailers leveraging data analytics achieve significantly higher profit margins compared to those relying on traditional reporting methods. In this project, retail analytics is applied to evaluate sales performance, customer segments, and profitability.

Chapter 3

Methodology

3.1 Research Approach

This project follows a quantitative, descriptive analytical approach based on the **Kimball Data Warehouse Lifecycle** [2], emphasising iterative development aligned with business requirements.

3.2 Data Collection

The **Superstore Sales Dataset** was obtained from Kaggle [6]:

<https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

The dataset consists of **9,994 records** and **21 attributes**. Key fields include Order ID, Order Date, Ship Date, Customer Segment, Product Category, Sales, Quantity, Discount, Profit, Region, State, and City. The dataset was selected because it contains realistic retail transaction data suitable for data warehousing, ETL processing, and Business Intelligence analysis.

3.3 Data Quality Analysis

Before loading the dataset into the data warehouse, a data quality assessment was conducted to identify inconsistencies and ensure data accuracy.

Table 3.1: Data Quality Assessment Results

Issue	Count	Action Taken
Missing Values	0	No action required
Duplicate Records	0	No action required
Inconsistent Date Formats	Present	Converted using <code>pd.to_datetime()</code>
Column Name Inconsistencies	Present	Standardized using <code>str.lower()</code>
Duplicate Product IDs	32	Removed using <code>drop_duplicates()</code>

3.4 Tools and Technologies Used

Table 3.2: Technologies and Tools Used

Category	Tool / Technology
Programming Language	Python 3.14
ETL Libraries	Pandas, os, sqlite3, datetime
Database	SQLite
BI and Visualization	Microsoft Power BI
Data Format	CSV (Comma Separated Values)
Development Environment	Visual Studio Code
Data Source	Kaggle — Superstore Sales Dataset [6]

Chapter 4

System Design and Implementation

4.1 Data Warehouse Architecture

The system adopts a multi-tier data warehouse architecture to support efficient data processing, storage, and analytical reporting [1]. The architecture consists of four tiers:

1. **Source Data Layer** — Raw Superstore Sales Dataset in CSV format (9,994 records) obtained from Kaggle.
2. **ETL Processing Layer** — Python and Pandas extract, clean, transform, and prepare data for loading into the warehouse.
3. **Data Warehouse Layer** — Transformed data loaded into a SQLite database using a Star Schema with one fact table and four dimension tables.
4. **Presentation Layer** — Microsoft Power BI dashboards and reports for data visualization and business analysis.

Figure 4.1 illustrates the overall system architecture.

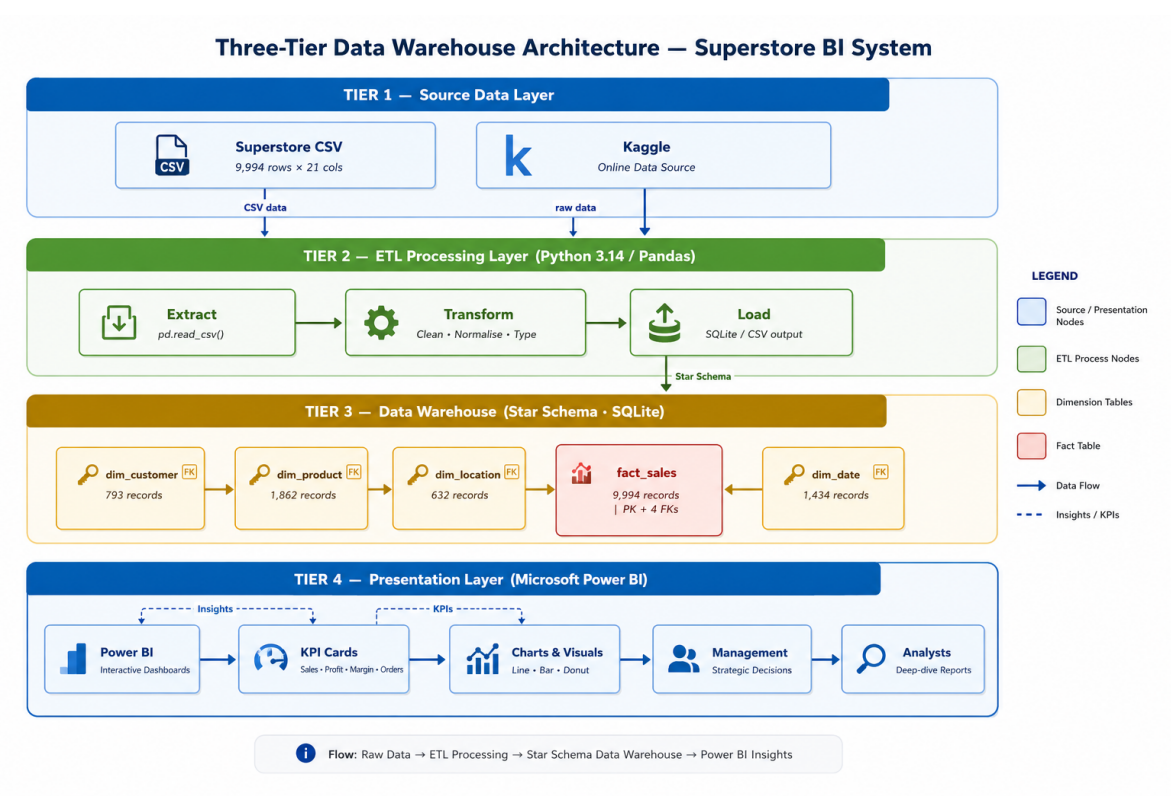


Figure 4.1: Three-Tier Data Warehouse Architecture — Superstore BI System

4.2 Data Flow Diagram (DFD)

Figure 4.2 presents the Level-1 DFD showing how data moves from the raw CSV source through the ETL pipeline into the data warehouse and finally to the Power BI presentation layer.

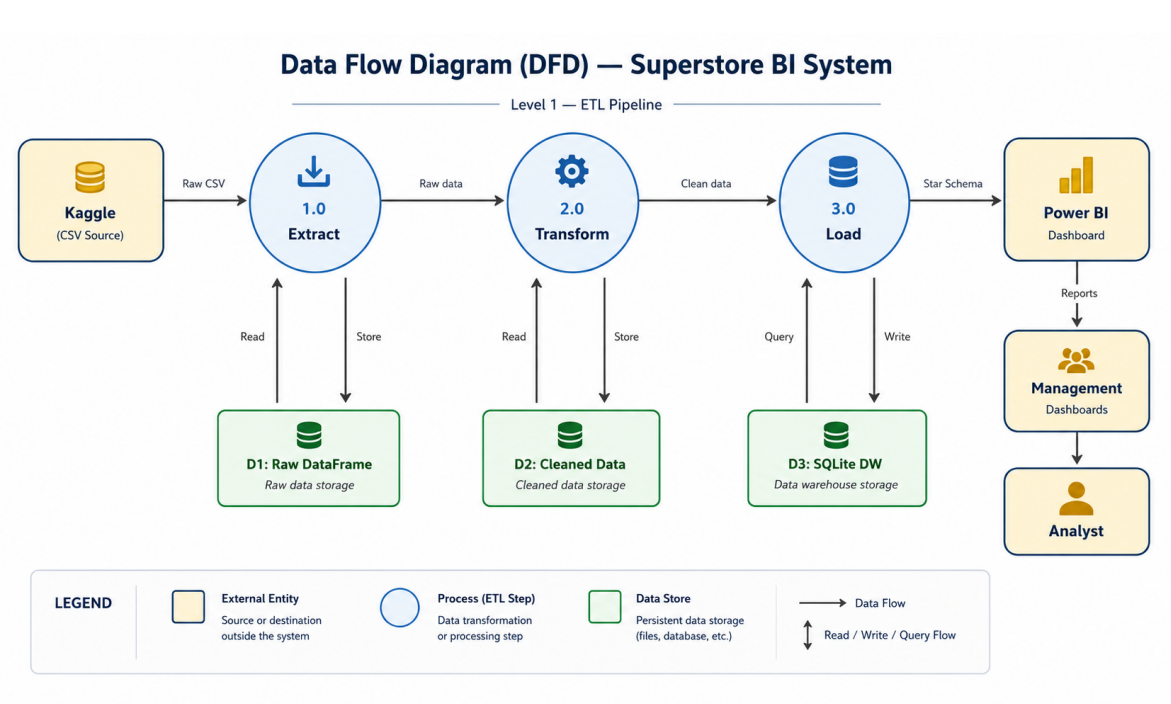


Figure 4.2: Level-1 Data Flow Diagram (DFD) — Superstore BI System

4.3 Star Schema Design

The Star Schema consists of one central fact table and four dimension tables following Kimball’s dimensional modelling principles [2]. Figure 4.3 shows the Power BI model view of the implemented schema.

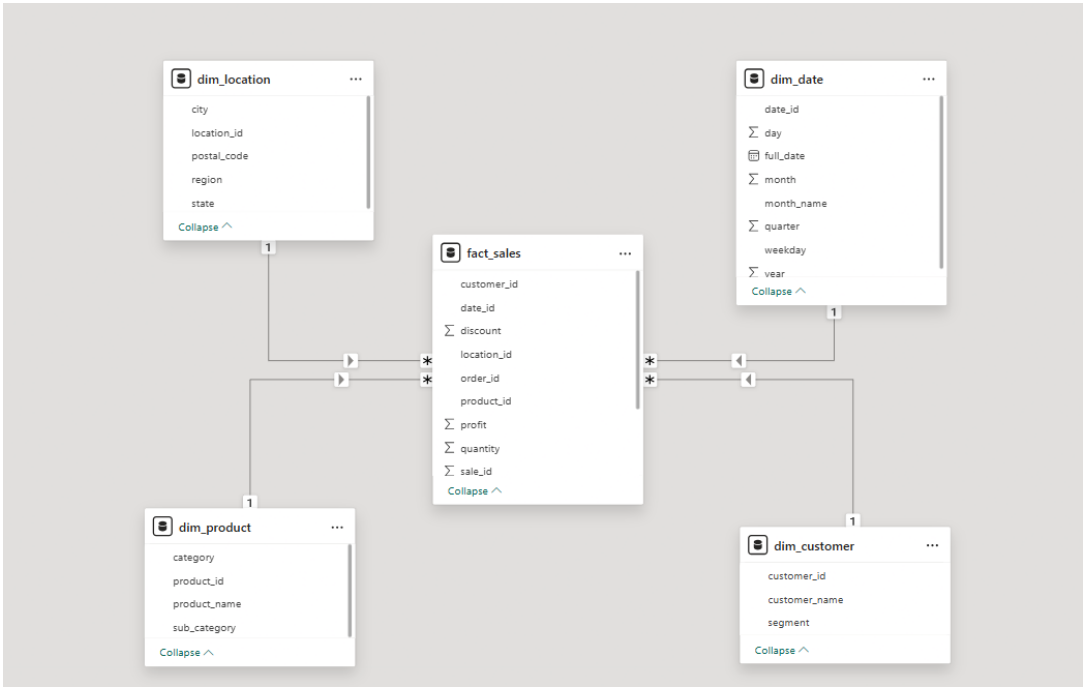


Figure 4.3: Star Schema — Power BI Model View

4.4 Entity Relationship Diagram (ERD)

Figure 4.4 presents the full ERD showing all tables, primary keys (PK), foreign keys (FK), and their many-to-one relationships.

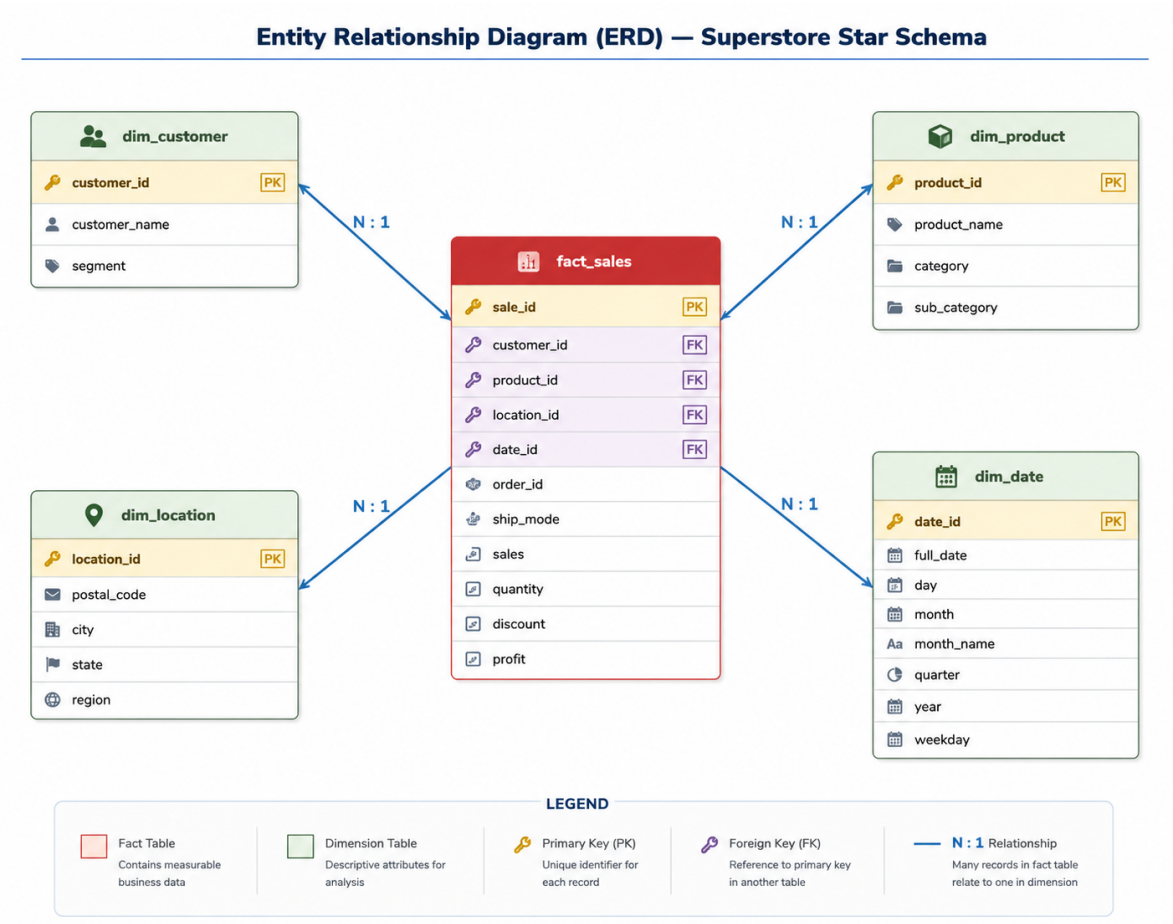


Figure 4.4: Entity Relationship Diagram (ERD) — Superstore Star Schema

4.4.1 Fact Table: fact_sales

Table 4.1: fact_sales Table Structure

Column Name	Data Type	Description
sale_id (PK)	INTEGER	Unique sale identifier (auto-generated)
order_id	VARCHAR	Order reference number
customer_id (FK)	VARCHAR	Links to dim_customer
product_id (FK)	VARCHAR	Links to dim_product
location_id (FK)	INTEGER	Links to dim_location
date_id (FK)	INTEGER	Links to dim_date (order date)
ship_mode	VARCHAR	Shipping method used
sales	DECIMAL	Revenue from the sale
quantity	INTEGER	Units sold
discount	DECIMAL	Discount rate applied
profit	DECIMAL	Profit generated

4.4.2 Dimension Tables

Table 4.2: Dimension Tables Summary

Table	Records	Key Columns
dim_customer	793	customer_id, customer_name, segment
dim_product	1,862	product_id, product_name, category, sub_category
dim_location	632	location_id, postal_code, city, state, region
dim_date	1,434	date_id, full_date, day, month, month_name, quarter, year, weekday

4.5 ETL Implementation

The ETL script (`etl_superstore.py`) follows four clearly defined phases: Extract, Transform, Star Schema construction, and Load. The complete script is provided in Appendix A.

4.5.1 Phase 1 — Extraction

```

1 import pandas as pd, os, sqlite3
2 from datetime import datetime
3
4 RAW_FILE = "superstore_raw.csv"
5 OUTPUT_DIR = "warehouse_output"
6 DB_FILE = f"{OUTPUT_DIR}/superstore_warehouse.db"
7 os.makedirs(OUTPUT_DIR, exist_ok=True)

```

```

8
9 try:
10     df = pd.read_csv(RAW_FILE, encoding="latin-1")
11 except FileNotFoundError:
12     print(f"ERROR: Could not find '{RAW_FILE}'.")
13     exit()
14
15 print(f"Loaded {len(df):,} rows and {len(df.columns)} columns.")

```

Listing 4.1: Data Extraction — loading the raw CSV file

4.5.2 Phase 2 — Transformation

```

1 df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
2 df.rename(columns={"sub-category": "sub_category"}, inplace=True)
3
4 df["order_date"] = pd.to_datetime(df["order_date"], dayfirst=False,
5     errors="coerce")
6 df["ship_date"] = pd.to_datetime(df["ship_date"], dayfirst=False,
7     errors="coerce")
8
9 df.drop_duplicates(inplace=True)
10 df.dropna(subset=["order_id", "customer_id", "product_id",
11     "order_date", "sales"], inplace=True)
12
13 df["postal_code"] = df["postal_code"].fillna("UNKNOWN").astype(str)
14 df["sales"] = pd.to_numeric(df["sales"], errors="coerce").
15     fillna(0).round(2)
16 df["profit"] = pd.to_numeric(df["profit"], errors="coerce").
17     fillna(0).round(2)
18 df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce").
19     fillna(0).astype(int)
20 df["discount"] = pd.to_numeric(df["discount"], errors="coerce").
21     fillna(0).round(2)
22
23 for col in ["customer_name", "segment", "city", "state", "region",
24     "category", "sub_category", "product_name"]:
25     if col in df.columns:
26         df[col] = df[col].str.strip()

```

Listing 4.2: Data Transformation — cleaning and standardising

4.5.3 Phase 3 — Star Schema Table Construction

```

1 dim_customer = df[["customer_id", "customer_name", "segment"]]\
2                 .drop_duplicates().reset_index(drop=True)
3
4 dim_product   = df[["product_id", "product_name", "category", "
5                     sub_category"]]\
6                 .drop_duplicates().reset_index(drop=True)
7
8 dim_location  = df[["postal_code", "city", "state", "region"]]\
9                 .drop_duplicates().reset_index(drop=True)
10
11 dim_location.insert(0, "location_id", range(1, len(dim_location)+1)
12                      )
13
14 all_dates = pd.concat([df["order_date"], df["ship_date"]])\
15                 .dropna().drop_duplicates()
16
17 dim_date = pd.DataFrame({"full_date": all_dates})\
18                 .drop_duplicates().reset_index(drop=True)
19
20 dim_date.insert(0, "date_id", range(1, len(dim_date)+1))
21
22 dim_date["day"]      = dim_date["full_date"].dt.day
23 dim_date["month"]    = dim_date["full_date"].dt.month
24 dim_date["month_name"] = dim_date["full_date"].dt.strftime("%B")
25 dim_date["quarter"]  = dim_date["full_date"].dt.quarter
26 dim_date["year"]     = dim_date["full_date"].dt.year
27 dim_date["weekday"]  = dim_date["full_date"].dt.strftime("%A")
28
29 df = df.merge(dim_location[["location_id", "postal_code",
30                             "city", "state", "region"]],
31               on=["postal_code", "city", "state", "region"], how="left")
32
33 df = df.merge(
34     dim_date[["date_id", "full_date"]].rename(
35         columns={"date_id": "order_date_id", "full_date": "order_date"}),
36     on="order_date", how="left")
37
38 fact_sales = df[["order_id", "customer_id", "product_id", "location_id",
39                  "order_date_id", "ship_mode", "sales", "quantity",
40                  "discount", "profit"]].copy()
41
42 fact_sales.rename(columns={"order_date_id": "date_id"}, inplace=True)

```



```

35 fact_sales = fact_sales.reset_index(drop=True)
36 fact_sales.insert(0, "sale_id", range(1, len(fact_sales)+1))

```

Listing 4.3: Building dimension and fact tables

4.5.4 Phase 4 — Loading into SQLite

```

1 dim_customer.to_csv(f"{OUTPUT_DIR}/dim_customer.csv", index=False)
2 dim_product.to_csv( f"{OUTPUT_DIR}/dim_product.csv", index=False)
3 dim_location.to_csv(f"{OUTPUT_DIR}/dim_location.csv", index=False)
4 dim_date.to_csv(    f"{OUTPUT_DIR}/dim_date.csv",      index=False
    )
5 fact_sales.to_csv(  f"{OUTPUT_DIR}/fact_sales.csv",     index=False
    )
6
7 conn = sqlite3.connect(DB_FILE)
8 dim_customer.to_sql("dim_customer", conn, if_exists="replace",
    index=False)
9 dim_product.to_sql( "dim_product", conn, if_exists="replace",
    index=False)
10 dim_location.to_sql("dim_location", conn, if_exists="replace",
    index=False)
11 dim_date.to_sql(    "dim_date",      conn, if_exists="replace",
    index=False)
12 fact_sales.to_sql(  "fact_sales",    conn, if_exists="replace",
    index=False)
13 conn.close()
14 print("ETL Complete. Database saved to:", DB_FILE)

```

Listing 4.4: Loading tables into SQLite and saving CSV files

4.6 Dashboard Development

The following DAX measures were created in Power BI:

```

1 Total Sales    = SUM(fact_sales[sales])
2 Total Profit   = SUM(fact_sales[profit])
3 Total Orders   = DISTINCTCOUNT(fact_sales[order_id])
4 Profit Margin  = DIVIDE(SUM(fact_sales[profit]),
5                          SUM(fact_sales[sales])) * 100
6 Monthly Sales = SUM(fact_sales[sales])

```

Listing 4.5: DAX Measures in Power BI

Figure 4.5 shows the completed Power BI dashboard filtered to 2017, displaying Total Sales of \$733.22K, Total Profit of \$93.44K, Profit Margin of 12.74, and 2K Total Orders.

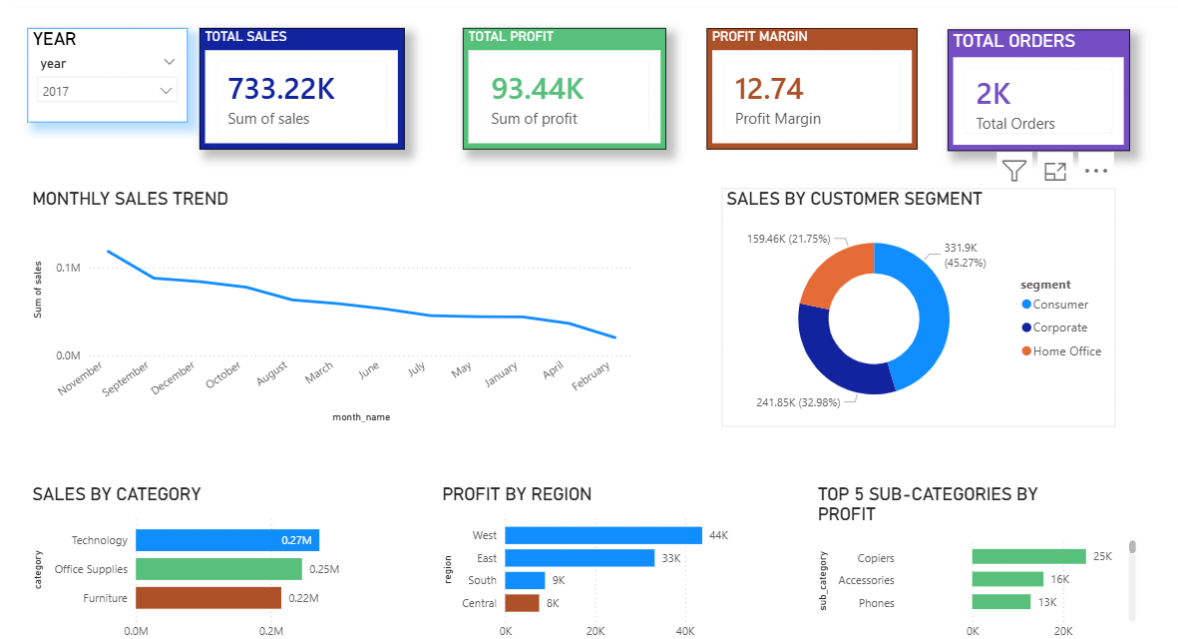


Figure 4.5: Power BI Dashboard — Superstore Sales Analytics (2017)

The dashboard includes the following visuals:

- **KPI Cards:** Total Sales, Total Profit, Profit Margin, Total Orders.
- **Line Chart:** Monthly Sales Trend using `dim_date[month_name]`.
- **Donut Chart:** Sales by Segment — Consumer (45.27%), Corporate (32.98%), Home Office (21.75%).
- **Bar Charts:** Sales by Category, Profit by Region, Top 5 Sub-Categories by Profit.
- **Year Slicer:** Dynamic cross-visual filter by year.

Chapter 5

Findings, Analysis, and Recommendations

5.1 Key Findings

5.1.1 Overall Sales Performance

The superstore generated **\$2.3M** total sales across 9,994 transactions with **\$286K** profit and **12.5%** margin overall. For 2017 specifically, sales were **\$733.22K** with **\$93.44K** profit. A total of **793 unique customers** placed orders during the analysis period.

5.1.2 Product Category Performance

Technology generated the highest profit margins. Office Supplies had the highest transaction volume. Furniture had the lowest margins due to aggressive discounting, as shown in Table 5.1.

Table 5.1: Sales and Profit by Product Category

Category	Total Sales	Total Profit
Technology	\$836,154	\$145,455
Furniture	\$741,999	\$18,451
Office Supplies	\$719,047	\$122,491

5.1.3 Regional Analysis

The West region led profitability (\$44K in 2017), followed by East (\$33K), South (\$9K), and Central (\$8K). Central's low profitability suggests excessive discounting or high operational costs.

5.1.4 Customer Segment Insights

The Consumer segment led with **45.27%** of sales, followed by Corporate (**32.98%**) and Home Office (**21.75%**). Corporate showed higher average order values making it a priority target segment.

5.1.5 Top Sub-Categories by Profit

The top three profitable sub-categories were Copiers (\$25K), Accessories (\$16K), and Phones (\$13K) — all from Technology, confirming it as the highest-yielding category.

5.1.6 Discount Impact

A **negative correlation** exists between discount rates and profit margins. Orders with discounts above 20% frequently resulted in negative profits, particularly in the Furniture category.

5.2 Recommendations

1. **Prioritise Technology:** Increase investment in Copiers, Phones, and Accessories which yield the highest profit margins [3].
2. **Review discounting:** Cap discounts at 15–20% to prevent profit erosion, particularly in Furniture.
3. **Target Corporate clients:** Develop loyalty programmes for high-value Corporate customers who generate higher order values.
4. **Investigate Central region:** Conduct cost analysis to address the lowest profitability region.
5. **Leverage seasonal trends:** Plan promotions and inventory ahead of Q4 peak periods.
6. **Replicate West region strategy:** Apply successful West region approaches across other regions to drive overall growth.

5.3 Conclusion

This project successfully implemented a complete BI solution for a retail superstore. The Python ETL pipeline processed 9,994 records into a Star Schema with four dimension tables stored in an SQLite database. Power BI dashboards provide actionable insights into sales, profitability, regional trends, and customer behaviour, demonstrating the practical value of Data Warehousing and Business Intelligence [2].

Bibliography

- [1] W. H. Inmon, *Building the Data Warehouse*, 4th ed. Hoboken, NJ: John Wiley & Sons, 2005.
- [2] R. Kimball and M. Ross, *The Data Warehouse Toolkit*, 3rd ed. Hoboken, NJ: John Wiley & Sons, 2013.
- [3] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.
- [4] Microsoft Corporation, “Power BI Documentation,” 2024. [Online]. Available: <https://docs.microsoft.com/en-us/power-bi/> [Accessed: May 2026].
- [5] Python Software Foundation, “Pandas Documentation,” 2024. [Online]. Available: <https://pandas.pydata.org/docs/> [Accessed: May 2026].
- [6] V. Kumar, “Superstore Dataset,” Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final> [Accessed: May 2026].

Appendices

Appendix A: Complete ETL Python Script

```
1 import pandas as pd
2 import os
3 import sqlite3
4 from datetime import datetime
5
6 RAW_FILE = "superstore_raw.csv"
7 OUTPUT_DIR = "warehouse_output"
8 DB_FILE = f"{OUTPUT_DIR}/superstore_warehouse.db"
9 os.makedirs(OUTPUT_DIR, exist_ok=True)
10
11 print("\n[EXTRACT] Loading raw data...")
12 try:
13     df = pd.read_csv(RAW_FILE, encoding="latin-1")
14 except FileNotFoundError:
15     print(f"ERROR: Could not find '{RAW_FILE}'.")
16     exit()
17 print(f"    Loaded {len(df):,} rows and {len(df.columns)} columns.")
18
19 print("[TRANSFORM] Cleaning data...")
20 df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
21 df.rename(columns={"sub-category": "sub_category"}, inplace=True)
22 df["order_date"] = pd.to_datetime(df["order_date"],
23                                   dayfirst=False, errors="coerce")
24 df["ship_date"] = pd.to_datetime(df["ship_date"],
25                                   dayfirst=False, errors="coerce")
26 print(f"    Duplicate rows : {df.duplicated().sum():,}")
27 print(f"    Missing values : {df.isnull().sum().sum():,}")
28 df.drop_duplicates(inplace=True)
29 df.dropna(subset=["order_id", "customer_id", "product_id",
30                  "order_date", "sales"], inplace=True)
31 df["postal_code"] = df["postal_code"].fillna("UNKNOWN").astype(str)
32 df["sales"] = pd.to_numeric(df["sales"], errors="coerce")\
33               .fillna(0).round(2)
34 df["profit"] = pd.to_numeric(df["profit"], errors="coerce")\
```

```

35         .fillna(0).round(2)
36 df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")\
37         .fillna(0).astype(int)
38 df["discount"] = pd.to_numeric(df["discount"], errors="coerce")\
39         .fillna(0).round(2)
40 for col in ["customer_name", "segment", "city", "state", "region",
41            "category", "sub_category", "product_name"]:
42     if col in df.columns:
43         df[col] = df[col].str.strip()
44 print("  Data cleaned successfully.\n")
45
46 print("[LOAD] Building Star Schema tables...")
47 dim_customer = df[["customer_id", "customer_name", "segment"]]\
48     .drop_duplicates().reset_index(drop=True)
49 print(f"  dim_customer   : {len(dim_customer):,} records")
50
51 dim_product  = df[["product_id", "product_name", "category", "
52     sub_category"]]\
53     .drop_duplicates().reset_index(drop=True)
54 print(f"  dim_product    : {len(dim_product):,} records")
55
56 dim_location = df[["postal_code", "city", "state", "region"]]\
57     .drop_duplicates().reset_index(drop=True)
58 dim_location.insert(0, "location_id", range(1, len(dim_location)+1)
59 )
60 print(f"  dim_location   : {len(dim_location):,} records")
61
62 all_dates = pd.concat([df["order_date"], df["ship_date"]])\
63     .dropna().drop_duplicates()
64 dim_date = pd.DataFrame({"full_date": all_dates})\
65     .drop_duplicates().reset_index(drop=True)
66 dim_date.insert(0, "date_id", range(1, len(dim_date)+1))
67 dim_date["day"] = dim_date["full_date"].dt.day
68 dim_date["month"] = dim_date["full_date"].dt.month
69 dim_date["month_name"] = dim_date["full_date"].dt.strftime("%B")
70 dim_date["quarter"] = dim_date["full_date"].dt.quarter
71 dim_date["year"] = dim_date["full_date"].dt.year
72 dim_date["weekday"] = dim_date["full_date"].dt.strftime("%A")
73 print(f"  dim_date       : {len(dim_date):,} records")
74
75 df = df.merge(

```

```

74     dim_location[["location_id", "postal_code", "city", "state", "
    region"]],
75     on=["postal_code", "city", "state", "region"], how="left")
76 df = df.merge(
77     dim_date[["date_id", "full_date"]].rename(
78         columns={"date_id": "order_date_id", "full_date": "order_date"
79     })),
79     on="order_date", how="left")
80
81 fact_sales = df[["order_id", "customer_id", "product_id", "location_id
    ",
82                 "order_date_id", "ship_mode", "sales", "quantity",
83                 "discount", "profit"]].copy()
84 fact_sales.rename(columns={"order_date_id": "date_id"}, inplace=True
    )
85 fact_sales = fact_sales.reset_index(drop=True)
86 fact_sales.insert(0, "sale_id", range(1, len(fact_sales)+1))
87 print(f"    fact_sales      : {len(fact_sales):,} records\n")
88
89 dim_customer.to_csv(f"{OUTPUT_DIR}/dim_customer.csv", index=False)
90 dim_product.to_csv( f"{OUTPUT_DIR}/dim_product.csv",   index=False)
91 dim_location.to_csv(f"{OUTPUT_DIR}/dim_location.csv",  index=False)
92 dim_date.to_csv(    f"{OUTPUT_DIR}/dim_date.csv",      index=False
    )
93 fact_sales.to_csv(  f"{OUTPUT_DIR}/fact_sales.csv",     index=False
    )
94
95 conn = sqlite3.connect(DB_FILE)
96 dim_customer.to_sql("dim_customer", conn, if_exists="replace",
    index=False)
97 dim_product.to_sql( "dim_product",   conn, if_exists="replace",
    index=False)
98 dim_location.to_sql("dim_location",  conn, if_exists="replace",
    index=False)
99 dim_date.to_sql(    "dim_date",      conn, if_exists="replace",
    index=False)
100 fact_sales.to_sql(  "fact_sales",    conn, if_exists="replace",
    index=False)
101 conn.close()
102
103 print("=" * 55)

```



```

104 print("ETL COMPLETE! Summary:")
105 print("=" * 55)
106 print(f"  dim_customer  -> {len(dim_customer):>5,} rows")
107 print(f"  dim_product    -> {len(dim_product):>5,} rows")
108 print(f"  dim_location    -> {len(dim_location):>5,} rows")
109 print(f"  dim_date        -> {len(dim_date):>5,} rows")
110 print(f"  fact_sales       -> {len(fact_sales):>5,} rows")
111 print(f"  SQLite DB: {DB_FILE}")
112 print("=" * 55)

```

Listing 5.1: etl_superstore.py — Full ETL Script

Appendix B: ETL Console Output

```

1 [EXTRACT] Loading raw data...
2   Loaded 9,994 rows and 21 columns.
3
4 [TRANSFORM] Cleaning data...
5   Duplicate rows   : 0
6   Missing values   : 0
7   Data cleaned successfully.
8
9 [LOAD] Building Star Schema tables...
10  dim_customer     :   793 records
11  dim_product      : 1,862 records
12  dim_location     :   632 records
13  dim_date         : 1,434 records
14  fact_sales       : 9,994 records
15
16 =====
17 ETL COMPLETE! Summary:
18 =====
19  dim_customer  ->   793 rows
20  dim_product   -> 1,862 rows
21  dim_location  ->   632 rows
22  dim_date      -> 1,434 rows
23  fact_sales    -> 9,994 rows
24  SQLite DB: warehouse_output/superstore_warehouse.db
25 =====

```

Listing 5.2: ETL Script Console Output

Appendix C: Star Schema Tables Summary

Table 5.2: Star Schema Tables Summary

Table Name	Records	Type
dim_customer	793	Dimension Table
dim_product	1,862	Dimension Table
dim_location	632	Dimension Table
dim_date	1,434	Dimension Table
fact_sales	9,994	Fact Table